

Estrutura de Repetição

Prof. Me. Alex Paixão

Lagarto – SE

Estruturas de Repetição

Roteiro:

- Introdução
- Comando `while`
- Comando `do...while`
- Op. de incremento; formas simplificadas
- Comando `for`

Estruturas de Repetição

Introdução:

- Estruturas Condicionais:
 - Novidade: Execução condicional de um bloco
- Estruturas de Repetição:
 - Novidade: Repetir a execução de um bloco
 - Controlado por condições
- Exemplos:
 - Preencher uma tabela
 - Aplicar operação a todos elementos da lista
 - Testar vários números
 - Percorrer matrizes, vetores, listas

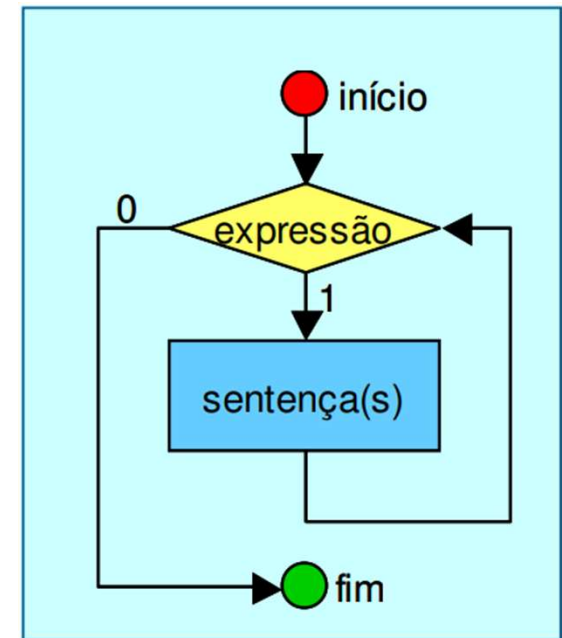
Estruturas de Repetição

Estrutura `while`

- Executa sentenças enquanto a condição for verdadeira.
- Condição é verificada antes do bloco.

Sintaxe:

```
início;  
while (expressão) {  
    sentença;  
    sentença;  
    ...  
}  
fim;
```



Estruturas de Repetição

Estrutura while

```
#include <iostream>
using namespace std;

int main() {
    int numero = 1;
    while (numero <= 10) {
        cout << numero << endl;
        numero = numero + 1;
    }
    return 0;
}
```

Inicializa valores
usados na condição

Condição de controle
da condição

Atualização dos
valores

1 2 3 4 5 6 7 8 9 10

Estruturas de Repetição

Estrutura while

```
#include <iostream>
using namespace std;
int main() {
    int numero, divisor, resto;
    cin >> numero;
    divisor = 1;
    while (divisor <= numero) {
        resto = numero % divisor
        if(resto == 0){
            cout << " Divisor: " << numero << endl;
        }
        divisor = divisor + 1;
    }
    return 0;
}
```

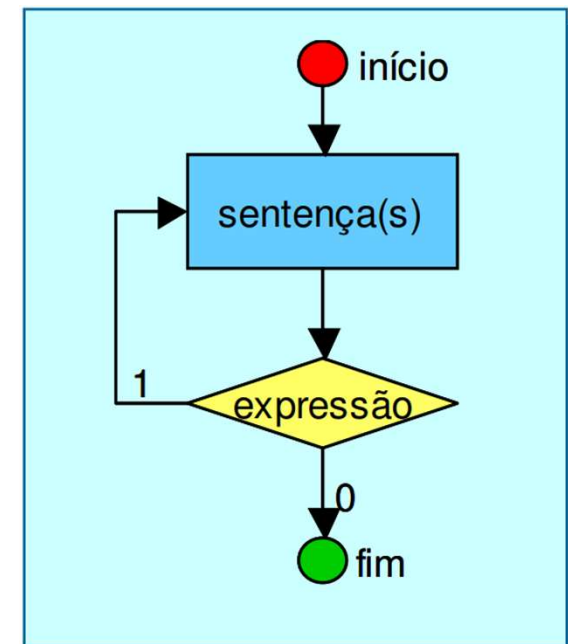
Estruturas de Repetição

Estrutura do . . . while

- Executa sentenças enquanto a condição for verdadeira.
- Condição é verificada depois do bloco.

Sintaxe:

```
início;  
do{  
    sentença;  
    sentença;  
    ...  
} while (expressao);  
fim;
```



Estruturas de Repetição

Estrutura do . . . while

```
#include <iostream>
using namespace std;

int main() {
    int numero = 1;
    do {
        cout << numero << endl;
        numero = numero + 1;
    } while (numero <= 10);
    return 0;
}
```

1 2 3 4 5 6 7 8 9 10

Estruturas de Repetição

Operadores de incremento:

- Antes:

`numero = numero + 1;`

`numero = numero - 1;`

Incrementa
Apresenta

- Agora:

`++numero;`

`--numero;`

- Retornam valor da variável **após** a operação

Estruturas de Repetição

Operadores de incremento:

- Antes:

`numero = numero + 1;`

`numero = numero - 1;`

Apresenta
Incrementa

- Agora:

`numero++;`

`numero--;`

- Retornam valor da variável **antes** a operação

Estruturas de Repetição

Operadores aritméticos: Notação simplificada

- Antes:

numero = numero * 10;

numero = numero + 3;

- Agora:

numero *=10;

numero +=3;

- Retornam valor da expressão

Estruturas de Repetição

Operadores de incremento:

Para:	Atalho:	Original:
Somar k unidades	<code>numero += k</code>	<code>numero=numero+k</code>
Subtrair k unidades	<code>numero -= k</code>	<code>numero=numero-k</code>
Multiplicar por k	<code>numero *= k</code>	<code>numero=numero*k</code>
Dividir por k	<code>numero /= k</code>	<code>numero=numero/k</code>

Estruturas de Repetição

Exemplo:

Antes

```
int numero = 1;
while (numero <= 10) {
    cout << numero;
    numero = numero + 1;
}
```

Depois

```
int numero = 1;
while (numero <= 10) {
    cout << numero;
    numero++;
}
```

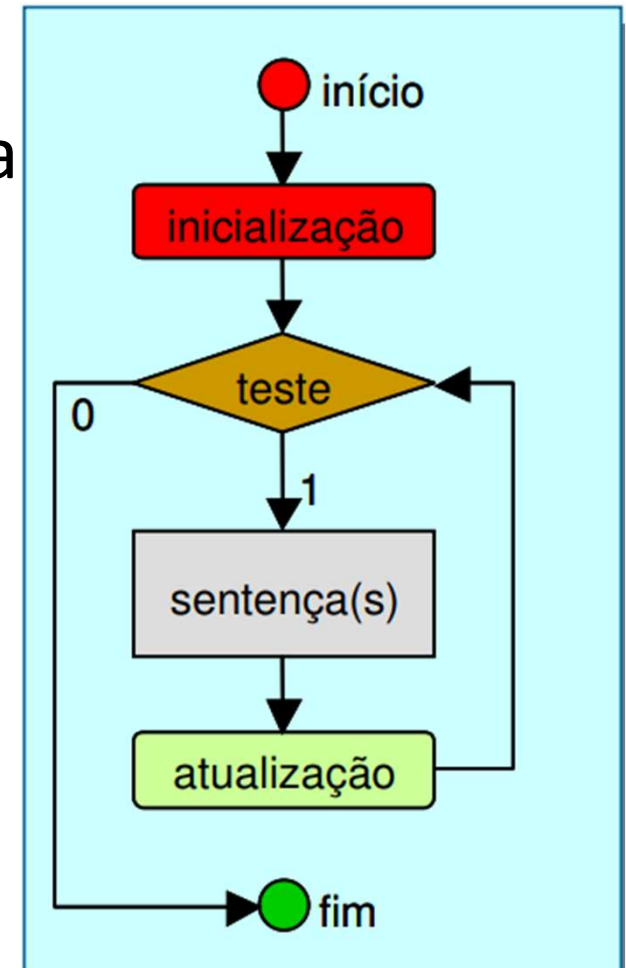
Estruturas de Repetição

Estrutura **for**:

- Automatiza estrutura de repetição típica

Sintaxe:

```
início;  
for (inicia ; teste ; atualização) {  
    sentença;  
    sentença;  
    ...  
}  
fim;
```



Estruturas de Repetição

Estrutura for

```
#include <iostream>
using namespace std;

int main() {
    int numero = 1;
    for(numero = 1 ; numero <= 10 ; numero++){
        cout << numero;
    }
    return 0;
}
```

1 2 3 4 5 6 7 8 9 10

Controle de Execução

Objetivo do `break`:

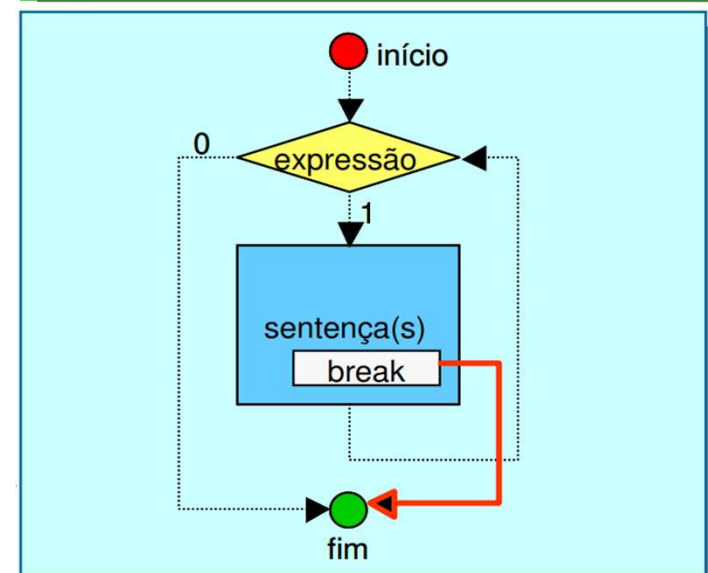
- **Comando** `break`
- **Comando** `continue`
- **Comando** `goto`

Controle de Execução

Objetivo do `break`:

- Cancelar execução:
 - `for` / `while` / `do...while`
- Comportamento:
 - Termina imediatamente o bloco
 - Não executa restante do bloco
 - Continua logo após o bloco
- Exemplos:
 - Terminar uma busca
 - Situações de erro
 - Evitar repetições

```
while (expressão) {  
    sentenças(s);  
    if (condição) {  
        break;  
    }  
    sentenças(s);  
}
```

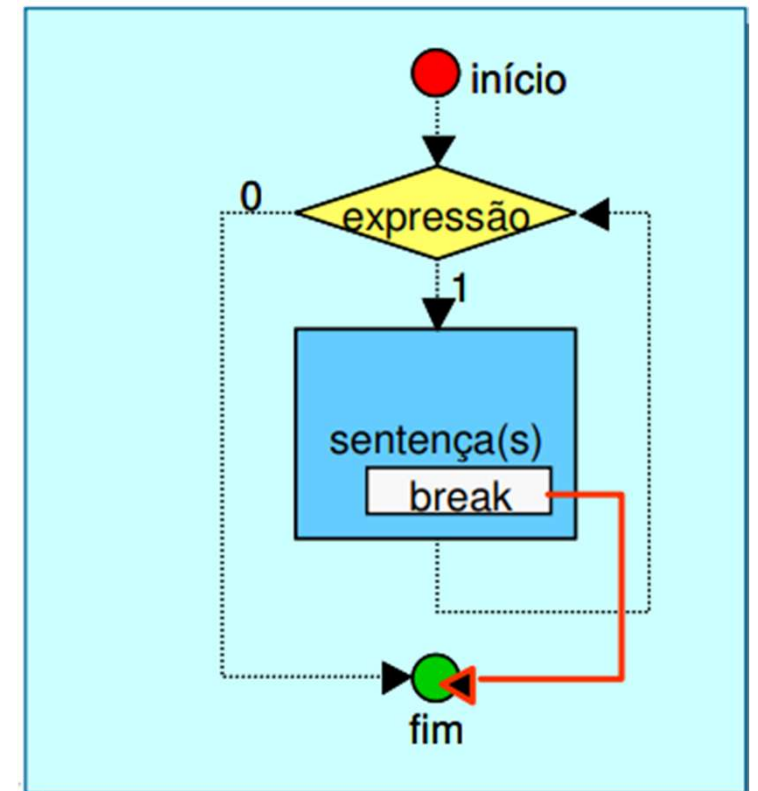


Controle de Execução

Sintaxe break com while

Sintaxe:

```
while (expressão) {  
    sentença(s) ;  
    if (condição) {  
        break ;  
    }  
}
```

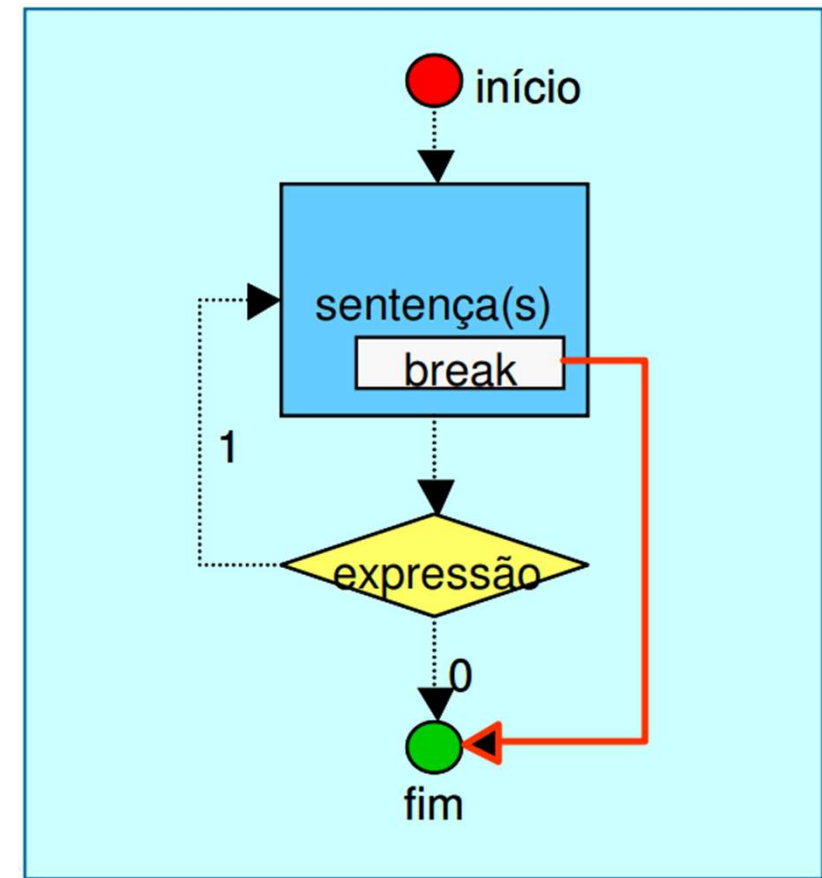


Controle de Execução

Sintaxe break com do...while

Sintaxe:

```
do {  
    sentença(s) ;  
    if (condição) {  
        break ;  
    }  
    sentença(s) ;  
} while (expressão) ;
```

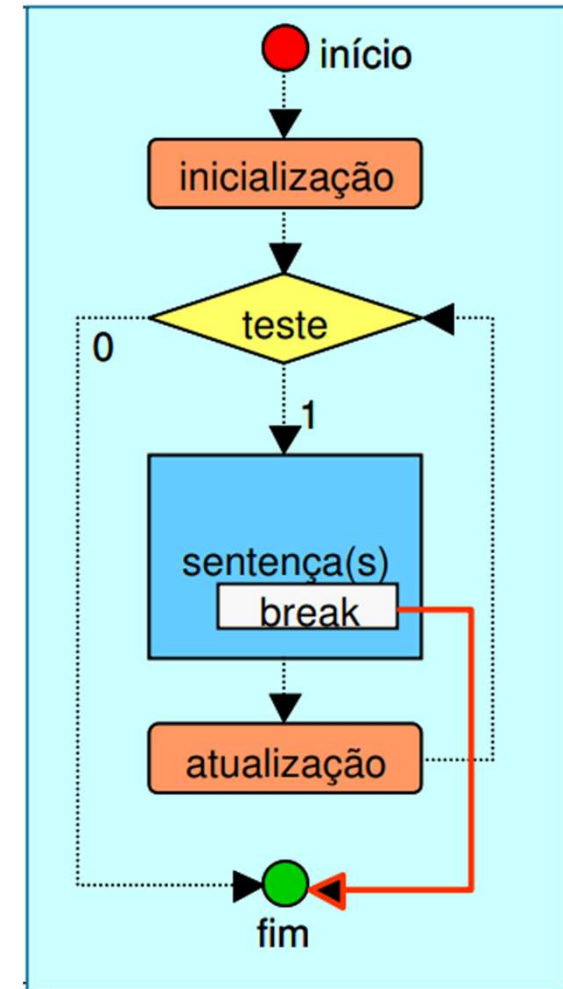


Controle de Execução

Sintaxe break com for ()

Sintaxe:

```
for (inicial ; teste ;  
atualização) {  
    sentença(s) ;  
    if (condição) {  
        break ;  
    }  
}
```



Controle de Execução

Estrutura for

```
#include <iostream>
using namespace std;

int main() {
    for(int i = 1; i <= 10; i++) {
        cout << i << endl;
        if(i == 5) {
            cout << "Break executado!" << endl;
            break;
        }
    }
    return 0;
}
```

1 2 3 4 5 Break executado!

Controle de Execução

Objetivo do `continue`:

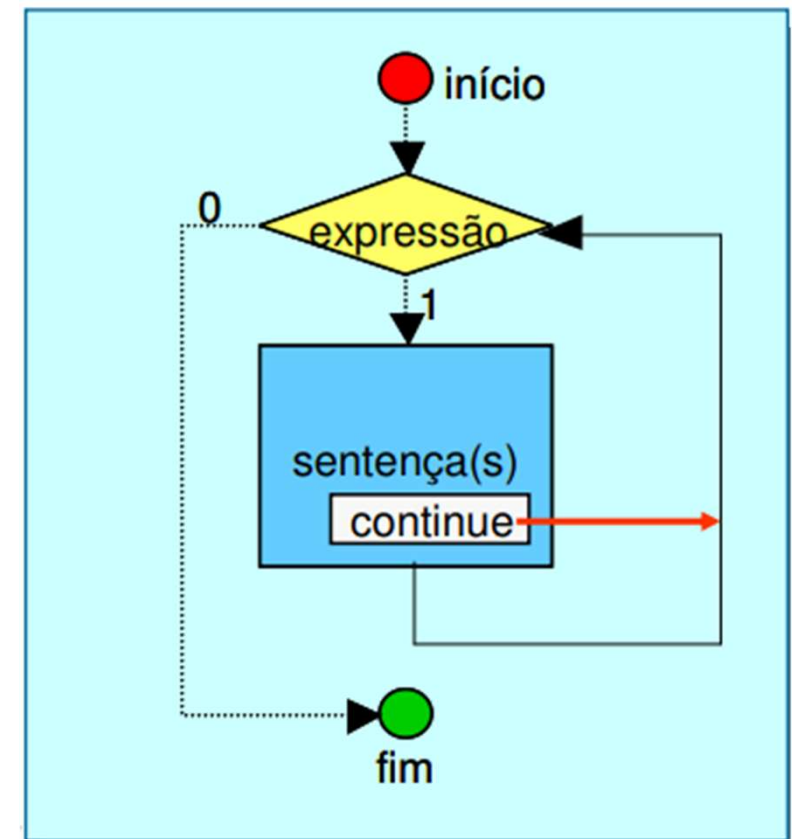
- Reiniciar execução:
 - `for` / `while` / `do...while`
- Comportamento:
 - Reinicia o bloco
 - Não executa resto do bloco
- Exemplos:
 - Pular valores inválidos
 - Evitar processamento

Controle de Execução

Sintaxe continue com while

Sintaxe:

```
while (expressão) {  
    sentença(s) ;  
    if (condição) {  
        continue;  
    }  
}
```

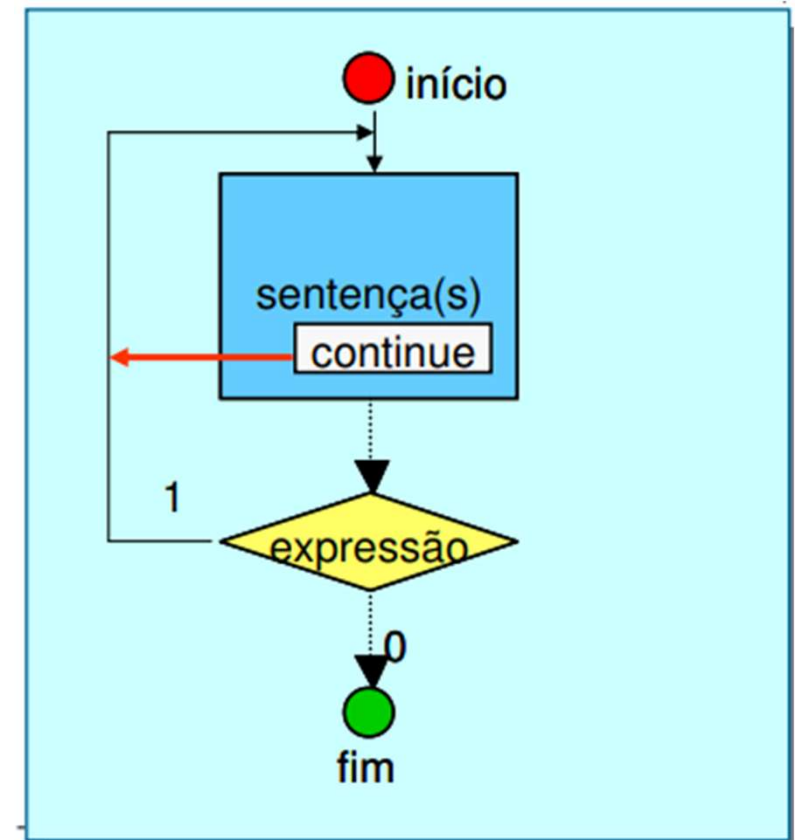


Controle de Execução

Sintaxe continue com do...while

Sintaxe:

```
do {  
    sentença(s) ;  
    if (condição) {  
        continue;  
    }  
    sentença(s) ;  
} while (expressão) ;
```



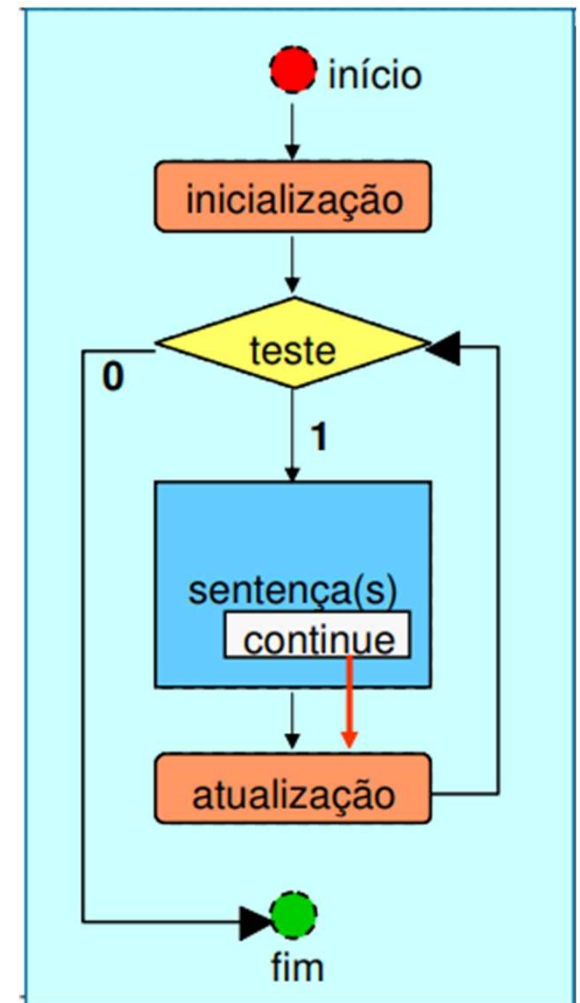
Controle de Execução

Sintaxe continue com for ()

Sintaxe:

```
for (inicial ; teste ;  
atualização) {  
    sentença(s) ;  
    if (condição) {  
        continue;  
    }  
}
```

OBS: executa também a atualização!



Controle de Execução

Estrutura for

```
#include <iostream>
using namespace std;

int main() {
    for(int i = 1; i <= 5; i++) {
        if(i == 3) {
            continue;
        }
        cout << i << endl;
    }
    return 0;
}
```

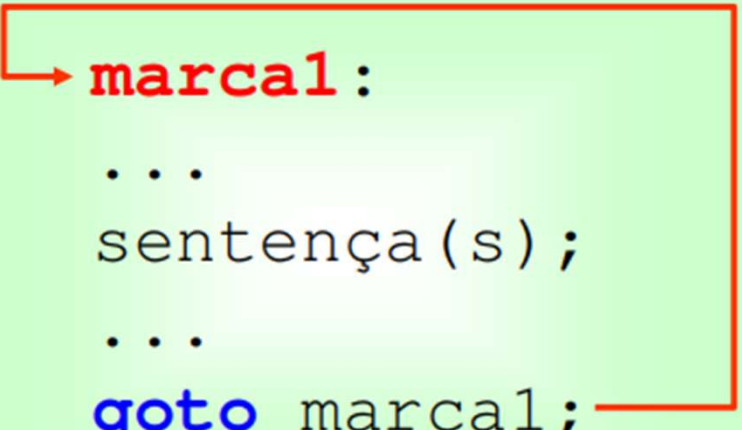
1 2 4 5

Controle de Execução

Objetivo do goto:

- Desviar execução para uma marca
- Saltos para pontos arbitrários
- Estrutura de repetição primitiva

Exemplo:
Repetição infinita



```
marcal :  
...  
sentença(s) ;  
...  
goto marca1;
```

The diagram illustrates an infinite loop structure using the goto statement. A red arrow originates from the 'goto marca1;' line and points back to the 'marcal :' label, indicating a jump to the start of the loop body.

Controle de Execução

Sintaxe goto

Retrocesso de execução:

Sintaxe:

```
sentença (s) ;  
...  
marca1:   
...  
sentença (s) ;  
...  
goto marca1;   
...  
sentença (s) ;
```

Avanço de execução:

Sintaxe:

```
sentença (s) ;  
...  
goto marca1:   
...  
sentença (s) ;  
...  
marca1;   
...  
sentença (s) ;
```

Controle de Execução

Uso do goto:

- Difícil visualizar os destinos do **goto**
- Oculta lógica de execução
- Programas tornam-se incompreensíveis!
- **Dica: não use goto**

